

D600 Task 2 - Aaron Pacheco

A. Describe the purpose of this data analysis by doing the following:

1. Propose **one** research question that is relevant to a real-world organizational situation captured in the provided dataset that you will answer using logistic regression in the initial model.

What factors play the biggest role in determining whether a house is considered luxury or not?

2. Define **one** goal of the data analysis. Ensure that your goal is relevant within the scope of the scenario and is represented in the available data.

This data analysis can help real estate agents determine what characteristics of a home are most likely to get the home labeled a luxury home. This can help with a variety of strategies, including pricing and marketing.

B. Summarize the data preparation process for logistic regression analysis by doing the following using Python or R:

1. Identify the dependent variable and **10** or more independent variables that are required to answer the research question and justify your selection of variables.

The dependent variable is "IsLuxury". It is a binary factor that tells us whether a home is considered luxury (1) or not (0). Our goal is to determine what characteristics are most likely to earn a home luxury status, so choosing this as our dependent variable is logical.

Independent Variables:

Square Footage, NumBathrooms, NumBedrooms, BackyardSpace - These four variables are directly tied to the size of a house. Logic seems to state that luxury homes are typically larger than non-luxury homes and contain more rooms.

CrimeRate, SchoolRating, DistanceToCityCenter, EmploymentRate - These four variables are tied to the area the house is in. It would be logical to believe that luxury homes are typically in nicer areas, so utilizing these four variables is a logical way to tie luxury status to location.

AgeOfHome, RenovationQualityRate - These two variables are tied to the quality of the build of the house. It seems logical that luxury properties would need to be well-maintained. These two variables should help determine if luxury status is tied to how old or how well renovated the home is.

2. Provide at least **one** screenshot of the descriptive statistics (e.g., mean, median, range, standard deviation, variance, percentiles, quartiles) for the quantitative (i.e.,

numeric) variables selected (e.g., frequency counts and percentages) from part B1 and for the qualitative (i.e., categorical) variables if selected.

```

D600 Task 1.py  D600 Task 2.py X  D600 Task 3.py
C: > Users > imret > Downloads > D600 Task 2.py > ...
25 #Calculate descriptive statistics
26 stats = df[description_vars].describe()
27
28 #Add additional statistics not included in describe()
29 stats.loc['median'] = df[description_vars].median()
30 stats.loc['variance'] = df[description_vars].var()
31 stats.loc['range'] = df[description_vars].max() - df[description_vars].min()
32
33 #Rename labels
34 stats = stats.rename(index={
35     'count': 'Count',
36     'mean': 'Mean',
37     'std': 'Standard Deviation',
38     'min': 'Minimum',
39     '25%': '25th Percentile (Q1)',
40     '50%': '50th Percentile (Q2)',
41     '75%': '75th Percentile (Q3)',
42     'max': 'Maximum',
43     'median': 'Median',
44     'variance': 'Variance',
45     'range': 'Range'
46 })
47
48 print(stats)

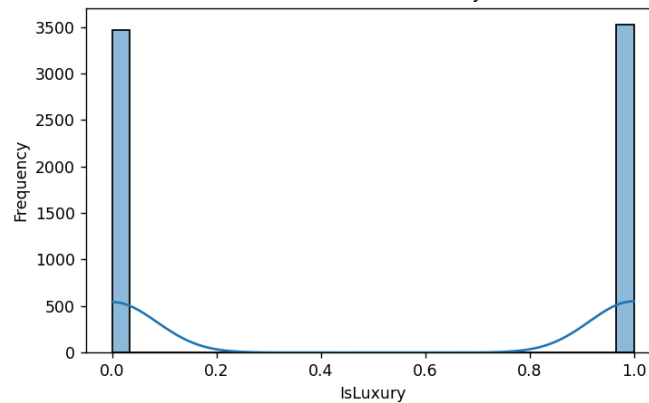
```

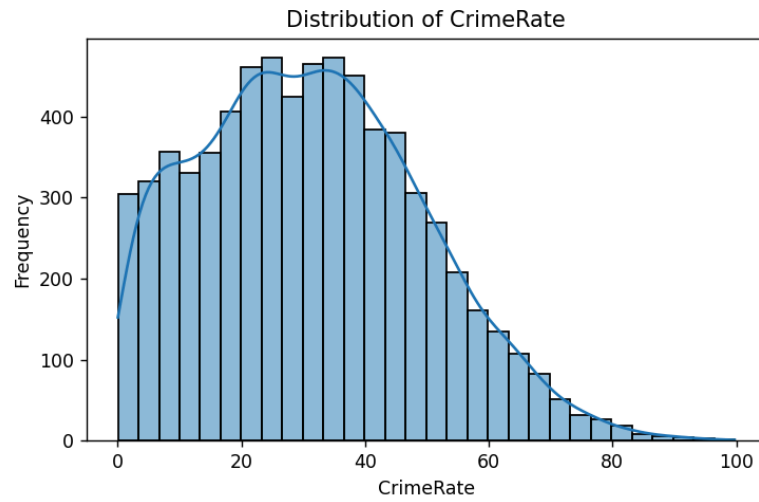
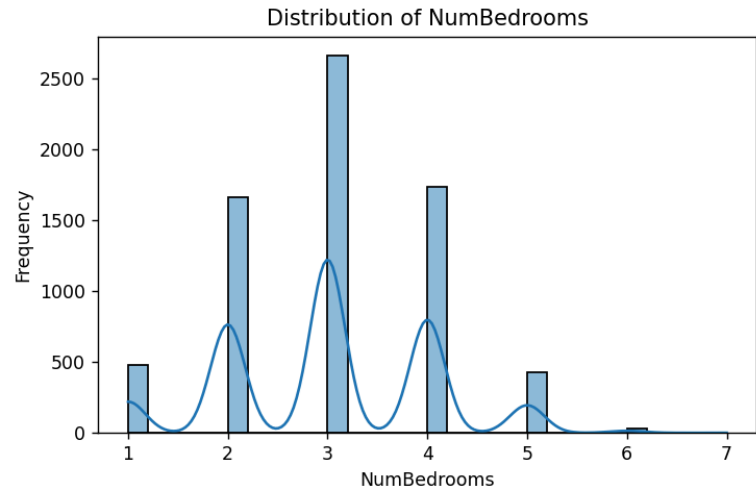
	ISLuxury	SquareFootage	NumBathrooms	NumBedrooms	BackyardSpace	CrimeRate	SchoolRating	AgeOfHome	DistanceToCityCenter	EmploymentRate	RenovationQualityRate
Count	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000
Mean	0.504002	1048.947459	2.117857	3.008571	511.507029	31.226194	6.949571	46.797046	17.475337	93.711340	5.016857
Standard Deviation	0.500002	426.010482	1.006889	1.021940	279.926549	18.025327	1.909931	31.779701	12.024985	4.585359	1.974766
Minimum	0.000000	550.000000	1.000000	1.000000	0.390000	0.030000	1.000000	0.010000	0.000000	72.050000	1.000000
25th Percentile (Q1)	0.000000	660.815000	1.000000	2.000000	300.995000	17.390000	6.000000	20.755000	7.827500	90.620000	4.000000
50th Percentile (Q2)	1.000000	996.320000	2.000000	3.000000	495.965000	30.385000	7.000000	42.620000	15.625000	94.010000	5.000000
75th Percentile (Q3)	1.000000	1342.292500	3.000000	4.000000	704.012500	43.670000	8.000000	67.232500	25.222500	97.410000	6.000000
Maximum	1.000000	2874.700000	6.000000	7.000000	1631.360000	99.730000	10.000000	178.600000	65.200000	99.900000	10.000000
Median	1.000000	996.320000	2.000000	3.000000	495.965000	30.385000	7.000000	42.620000	15.625000	94.010000	5.000000
Variance	0.250002	181484.930511	1.013826	1.044361	78358.872920	324.912428	3.647835	1009.949427	144.600276	20.298260	3.899702
Range	1.000000	2324.700000	5.000000	6.000000	1630.970000	99.700000	9.000000	178.670000	65.200000	27.850000	9.000000

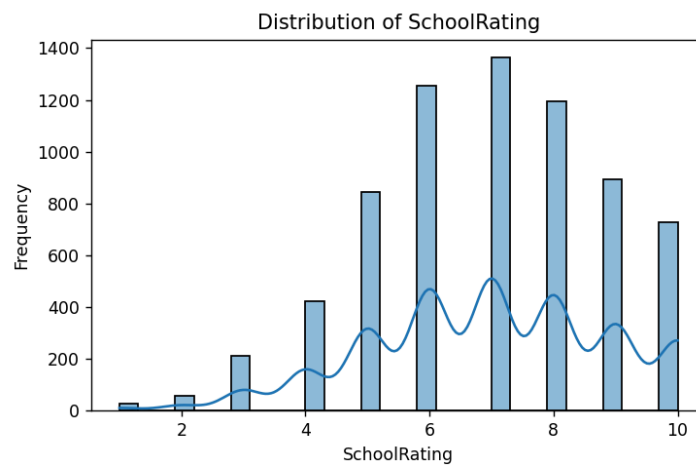
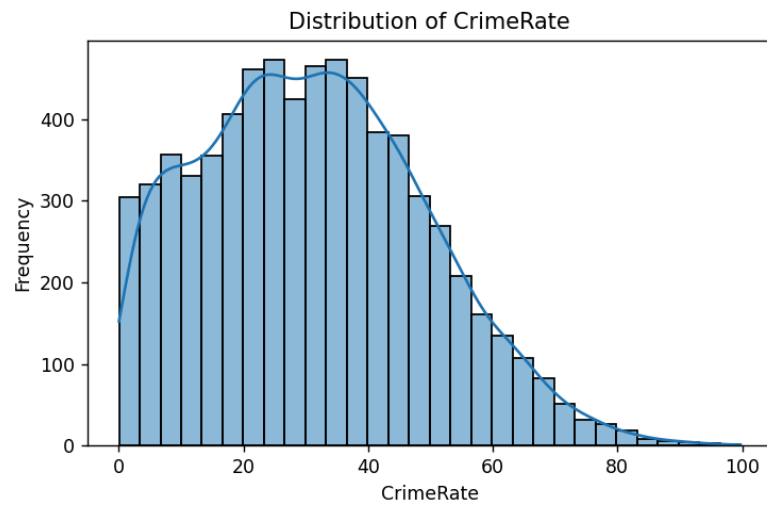
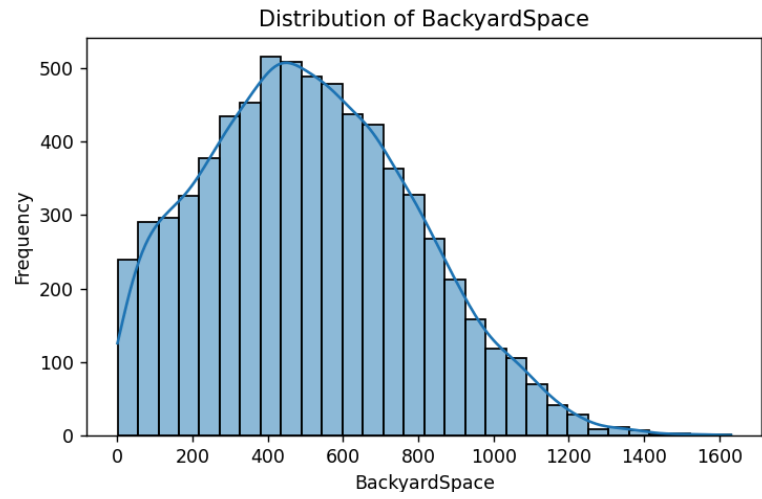
3. Generate univariate and bivariate visualizations of the distributions of the dependent and independent variables from part B1, including the dependent variable in the bivariate visualizations.

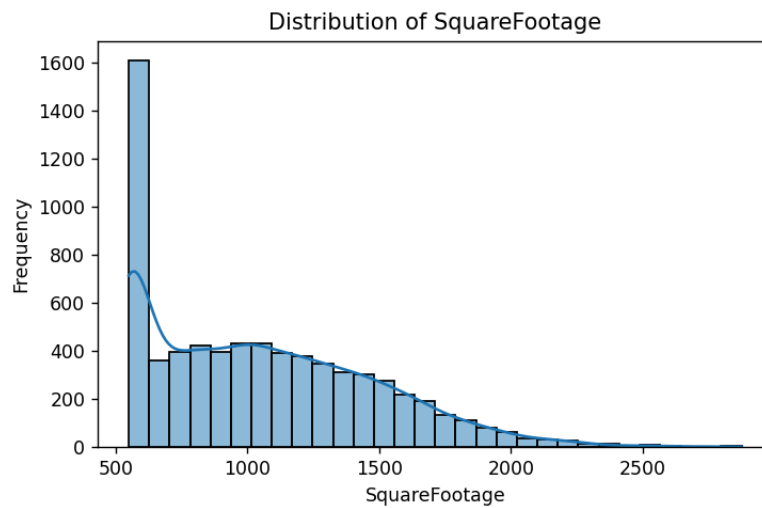
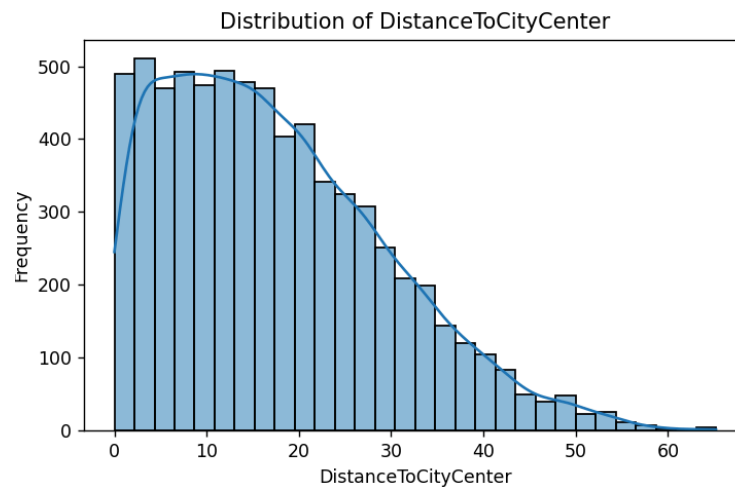
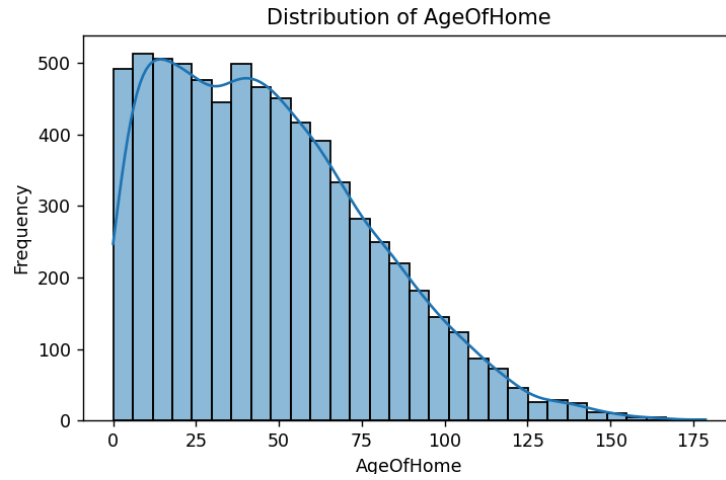
Univariate:

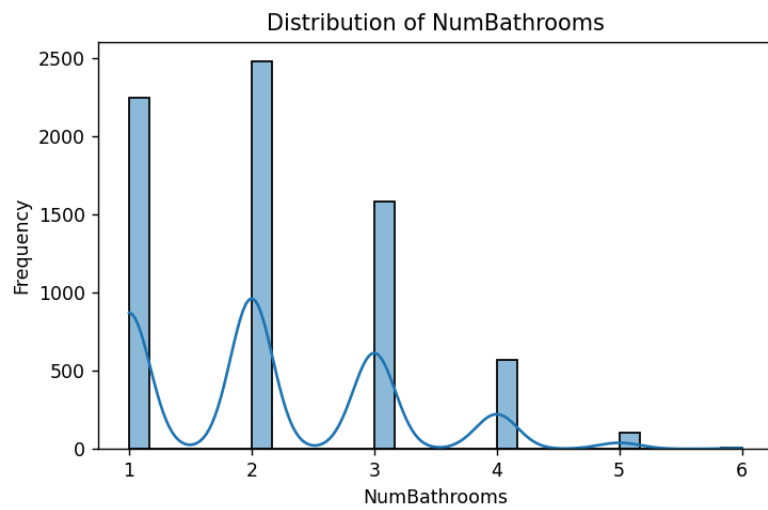
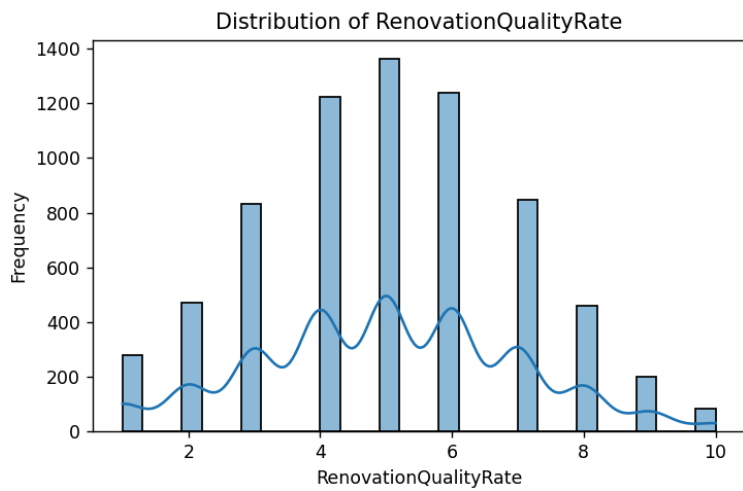
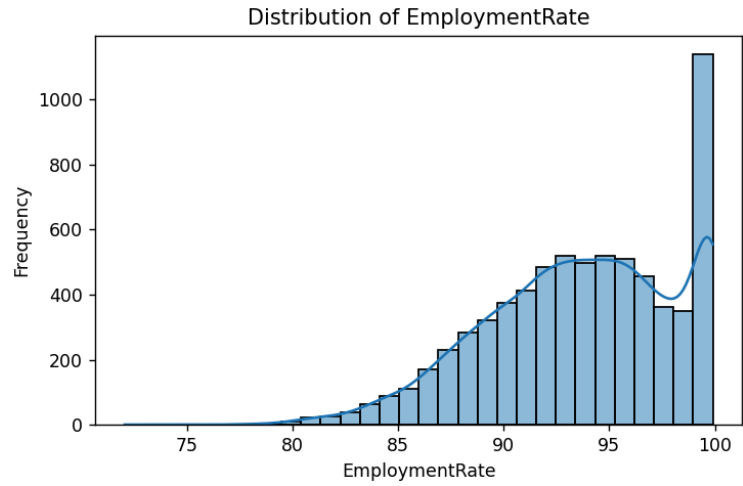
Distribution of ISLuxury



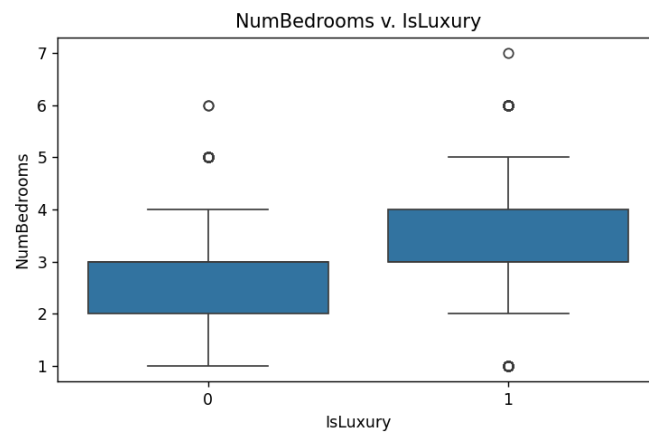
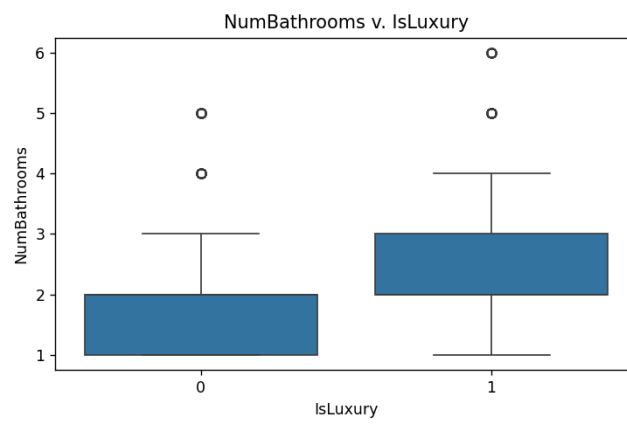
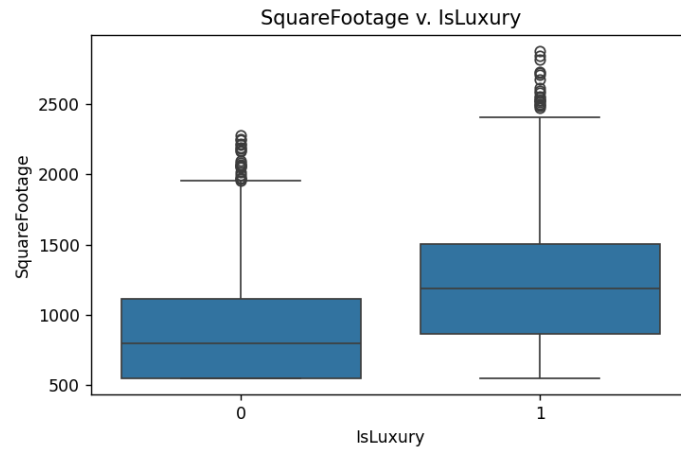


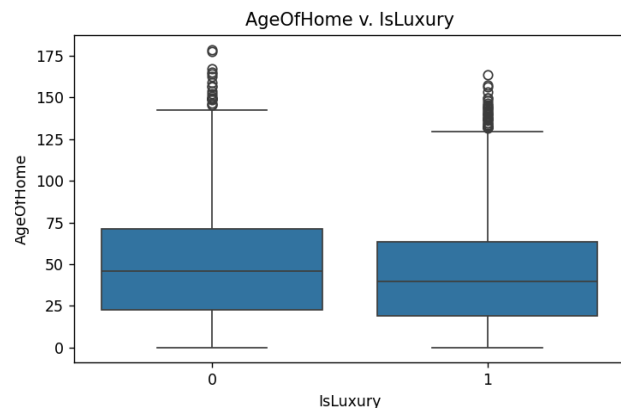
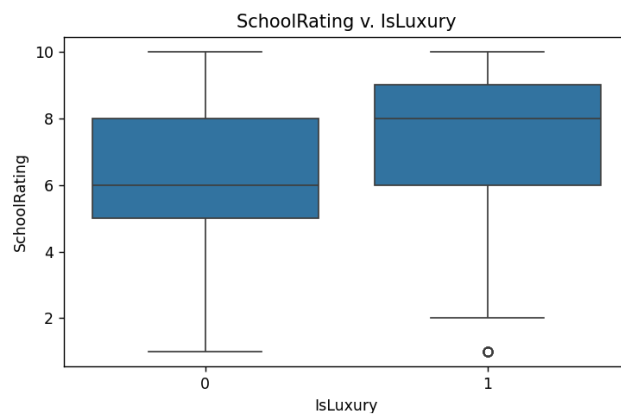
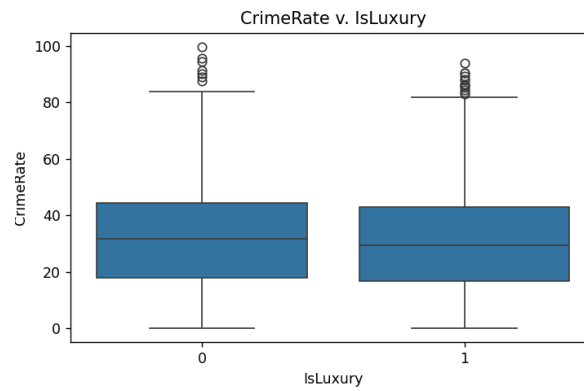
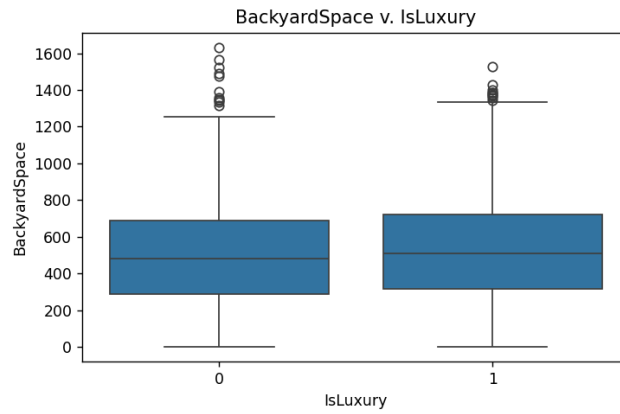


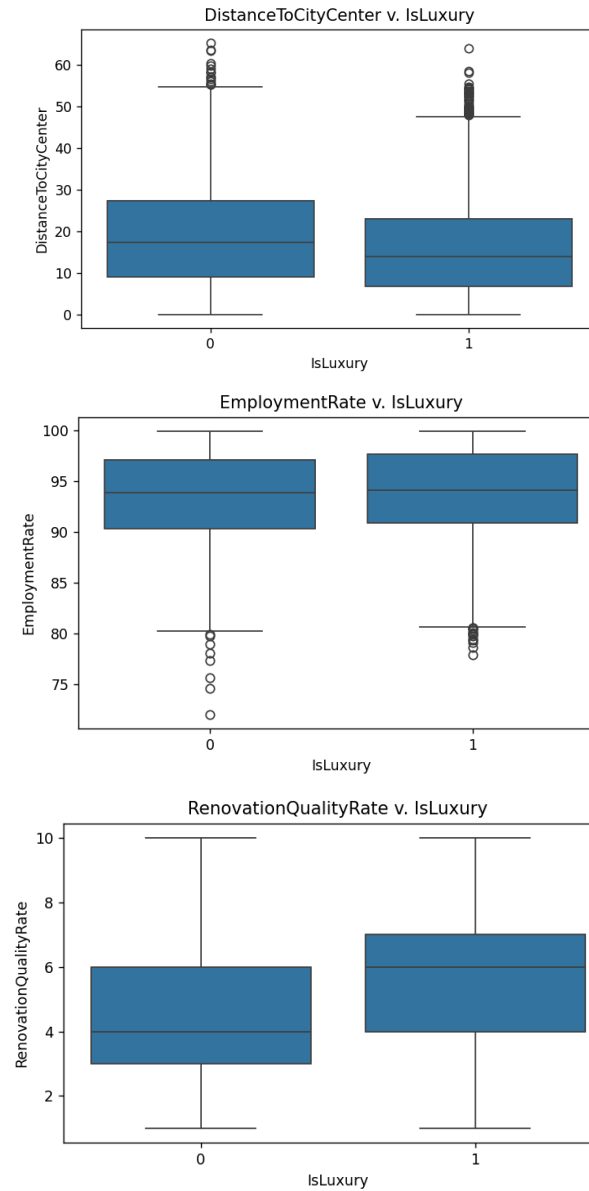




Bivariate:







4. Perform the following data cleaning techniques to prepare the data for modeling and provide a screenshot of the completed steps:

- Handling missing values
- Mitigating outliers
- Encoding categorical variables

```

74 #Drop rows with missing values
75 relevant_df=relevant_df.dropna()
76
77 #Identify outliers
78 col_mean = relevant_df.mean()
79 col_std = relevant_df.std()
80
81 zscore = (relevant_df-col_mean)/col_std
82
83 #Create dataframe without outliers
84 clean_df = relevant_df[(zscore.abs()< 3).all(axis=1)]

```

C. Perform the data analysis and report on the results by doing the following using Python or R:

- 1. Split the data into two datasets, with a larger percentage assigned to the training dataset and a smaller percentage assigned to the test dataset and include the files as instructed in part E4.**

Note: 70/30 or 80/20 are acceptable percentages for splitting the data.

```

#Split data into training set and test set
from sklearn.model_selection import train_test_split
train_df, test_df = train_test_split(clean_df, test_size=0.2, random_state=83)

#Defining independent inputs and dependent output in training set
x_train = train_df.drop(columns=['IsLuxury'])
y_train = train_df['IsLuxury']

#Defining independent inputs and dependent output in training set
x_test = test_df.drop(columns=['IsLuxury'])
y_test = test_df['IsLuxury']

```

- 2. Use the training dataset to create a model using logistic regression as a statistical method.*

```

#Perform Logistic Regression to Build Model
features = [
    'SquareFootage',
    'NumBathrooms',
    'NumBedrooms',
    'BackyardSpace',
    'CrimeRate',
    'SchoolRating',
    'AgeOfHome',
    'DistanceToCityCenter',
    'EmploymentRate',
    'RenovationQualityRate'
]
x_train_const = sm.add_constant(x_train[features])
logit_model = sm.Logit(y_train, x_train_const)
result = logit_model.fit(max_iter=1000)

print(result.summary())
print(f"AIC: {result.aic:.4f}")
print(f"BIC: {result.bic:.4f}")

```

3. Create a screenshot of the summary of the initial logistic regression model or the following extracted model parameters:
- AIC
 - BIC
 - pseudo R²
 - coefficient estimates
 - p-value of each independent variable

Logit Regression Results						
=====						
Dep. Variable:	IsLuxury	No. Observations:	5451			
Model:	Logit	Df Residuals:	5440			
Method:	MLE	Df Model:	10			
Date:	Tue, 07 Apr 2026	Pseudo R-squ.:	0.2242			
Time:	08:35:06	Log-Likelihood:	-2931.1			
converged:	True	LL-Null:	-3778.2			
Covariance Type:	nonrobust	LLR p-value:	0.000			
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-5.6697	0.717	-7.906	0.000	-7.075	-4.264
SquareFootage	0.0018	8.94e-05	20.352	0.000	0.002	0.002
NumBathrooms	0.5361	0.036	15.102	0.000	0.467	0.606
NumBedrooms	0.6875	0.037	18.734	0.000	0.616	0.759
BackyardSpace	-5.991e-05	0.000	-0.519	0.604	-0.000	0.000
CrimeRate	-0.0004	0.002	-0.199	0.842	-0.004	0.003
SchoolRating	-0.0158	0.021	-0.759	0.448	-0.057	0.025
AgeOfHome	-0.0016	0.001	-1.535	0.125	-0.004	0.000
DistanceToCityCenter	-0.0027	0.003	-0.981	0.327	-0.008	0.003
EmploymentRate	0.0028	0.007	0.377	0.706	-0.012	0.017
RenovationQualityRate	0.1230	0.020	6.127	0.000	0.084	0.162
=====						
AIC: 5884.1069						
BIC: 5956.7460						

4. Optimize the regression model using a process of your selection (e.g., backward stepwise elimination, recursive selection, etc.).

```

def backward_stepwise(x, y):
    cols = list(x.columns)

    while True:
        # Fit the model
        x_sm = sm.add_constant(x[cols])
        model = sm.Logit(y, x_sm).fit(dis=0)

        # Get p-values, excluding the constant
        pvalues = model.pvalues.drop('const')
        max_pvalue = pvalues.max()

        # If all p-values are below 0.05, stop
        if max_pvalue < 0.05:
            print("Final model reached.")
            break

        # Otherwise, remove the variable with the highest p-value
        removed = pvalues.idxmax()
        print(f"Removing: {removed} (p-value: {max_pvalue:.4f})")
        cols.remove(removed)

    print(model.summary())
    print(f"AIC: {model.aic:.4f}")
    print(f"BIC: {model.bic:.4f}")
    return model

# Perform backward stepwise elimination on training data
final_model = backward_stepwise(x_train, y_train)

```

5. Provide a screenshot of the summary of the optimized model or the following extracted model parameters:

- AIC
- BIC
- pseudo R²
- coefficient estimates
- p-value of each independent variable

Logit Regression Results						
=====						
Dep. Variable:	IsLuxury	No. Observations:	5451			
Model:	Logit	Df Residuals:	5446			
Method:	MLE	Df Model:	4			
Date:	Tue, 07 Apr 2026	Pseudo R-squ.:	0.2236			
Time:	09:25:57	Log-Likelihood:	-2933.3			
converged:	True	LL-Null:	-3778.2			
Covariance Type:	nonrobust	LLR p-value:	0.000			
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-5.6621	0.174	-32.631	0.000	-6.002	-5.322
SquareFootage	0.0018	8.81e-05	20.662	0.000	0.002	0.002
NumBathrooms	0.5363	0.035	15.268	0.000	0.467	0.605
NumBedrooms	0.6830	0.035	19.488	0.000	0.614	0.752
RenovationQualityRate	0.1222	0.019	6.512	0.000	0.085	0.159
=====						
AIC: 5876.5199						
BIC: 5909.5377						

6. Run the prediction on the test dataset using the variables from the optimized regression model from part C4 to create a confusion matrix. Provide a screenshot of the confusion matrix and accuracy results.

```
#Add constant to test data and only include attributes deemed relevant following backward elimination
x_test_const = sm.add_constant(x_test[['SquareFootage', 'NumBathrooms', 'NumBedrooms', 'RenovationQualityRate']])

#Get probabilities
y_prob = final_model.predict(x_test_const)

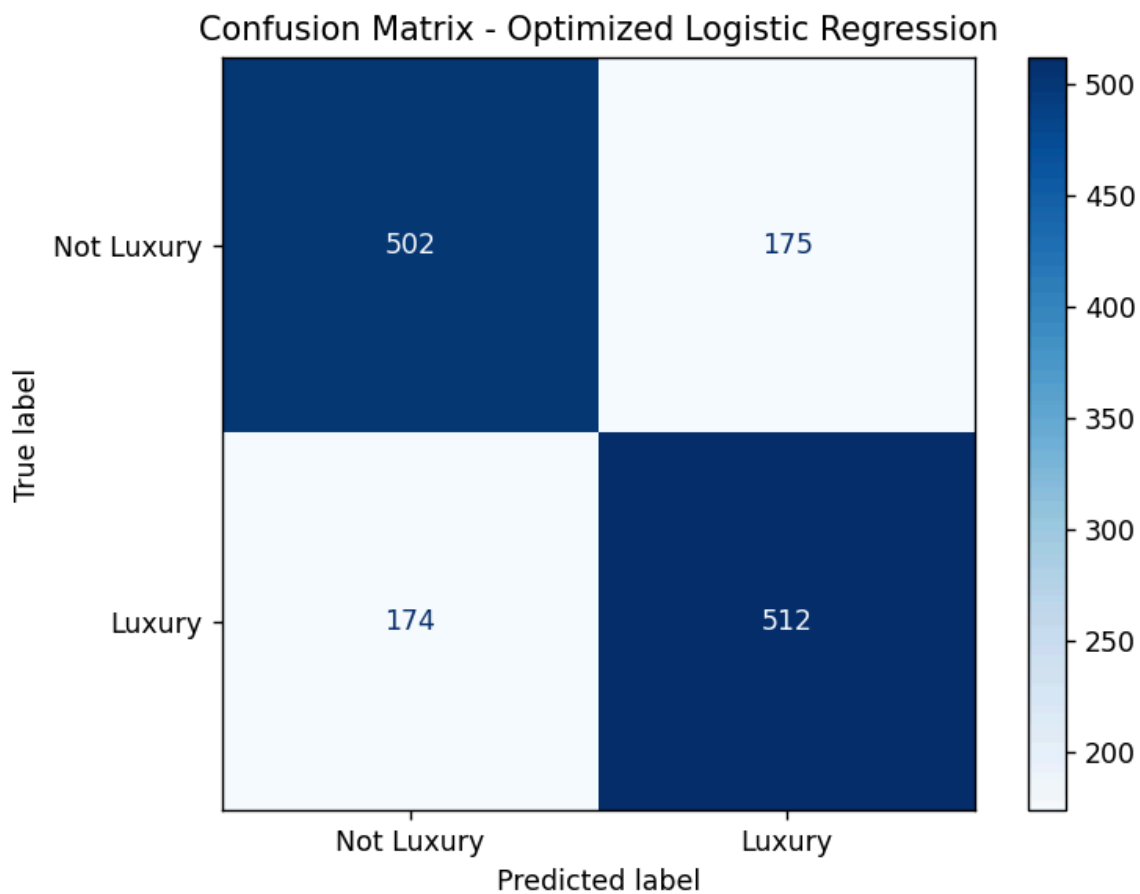
#Convert probabilities to binary outputs
y_pred = (y_prob >= 0.5).astype(int)

#Create Confusion Matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix = cm,
    display_labels = ['Not Luxury', 'Luxury']
)

disp.plot(cmap = 'Blues')
plt.title('Confusion Matrix - Optimized Logistic Regression')
plt.tight_layout()
plt.show()
```



```
# Accuracy Results
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred, target_names=['Not Luxury', 'Luxury']))
```

	precision	recall	f1-score	support
Not Luxury	0.74	0.74	0.74	677
Luxury	0.75	0.75	0.75	686
accuracy			0.74	1363
macro avg	0.74	0.74	0.74	1363
weighted avg	0.74	0.74	0.74	1363

D. Summarize your data analysis by doing the following:

1. List the packages or libraries you have chosen for Python or R and justify how each item supports the analysis.

Pandas - Used to convert the csv file into a dataframe we can work with in python

Matplotlib.pyplot - Used to generate structures for all univariate and bivariate visualizations

Seaborn - Takes the structures created by Matplotlib and turns them into visually appealing graphics.

Numpy - Used to run Box-Tidwell Test.

Statsmodels.api - Runs our regression model and our backward stepwise optimization

statsmodels.stats.outliers_influence(variance_inflation_factor) - Used to find the variance inflation factor to determine no multicollinearity.

Sklearn.model_selection (train_test_split) - Used to split our data into a training set and a test set

Sklearn.metrics (confusion_matrix & ConfusionMatrixDisplay & classification_report) -

Used to build the structure and graphic for the confusion matrix of our optimized logistic regression and create the classification report on our results to look for accuracy.

2. Discuss the method used to optimize the model and justify the approach.

Backward stepwise elimination was used to optimize our model. Since our primary goal is determining what factors contribute to a property's luxury status, this method of optimization is great because it is reliant on each variable's p-value and eliminates variables that don't play a strong role in determining luxury status. It was also well-suited to optimizing our data, where 6 of our 10 variables had pretty high p-values and we were able to improve interpretability by reducing the variables from 10 to 4.

3. Complete the following for the optimized model:

a. Create a regression equation by using the coefficients from the optimized model.

Based on the coefficients of our model, the regression equation would be

$$\ln(p/(1-p)) = -5.6621 + 0.0018(\text{SquareFootage}) + 0.5363(\text{NumBathrooms}) + 0.6830(\text{NumBedrooms}) + 0.1222(\text{RenovationQualityRate})$$

b. Provide an interpretation for each coefficient in the regression equation.

The coefficients provide the log-odds of each factor. You can determine the true event odds of each factor by exponentiating the log-odds.

Square Footage: 0.0018 exponentiates to 1.0018

NumBathrooms: 0.5363 exponentiates to 1.7098

NumBedrooms: 0.6830 exponentiates to 1.9803

RenovationQualityRate: 0.1222 exponentiates to 1.1300

So this tells us that each square foot increases the odds of a house being considered luxury by 0.18%, each additional bathroom increases the odds by 70.98%, each

additional bathroom increases the odds by 98.03%, and each point increase to the RenovationQualityRate increases the odds by 13%.

Exponentiating the intercept of the equation also provides us the baseline odds of a house being luxury.

-5.6621 exponentiates to 0.00346

This tells us that a house has baseline odds of about 0.35% to be considered luxury, which makes sense since a house with no square footage, no bathrooms, and no bedrooms would obviously never be considered luxury.

c. Discuss the pseudo R^2 .

The Pseudo R^2 value of the model explains how much the model's predictors improved the fit over a model that predicts nothing. The pseudo R^2 value of my optimized model was 0.2236. This tells us that the optimized model's predictors improved the fit by 22.36%. This value means the model is a good fit and does a reasonable job of explaining what makes a property "luxury" based on 4 variables.

d. Discuss the accuracy of the optimized model with that of the test dataset based on the confusion matrix.

The confusion matrix shows us that of all properties that the model predicted as "Not Luxury", 74% were actually not luxury. (502/676)

It also shows that 75% of the properties predicted to be luxury were actually luxury. (512/687)

This 74% accuracy means that our model is 24% more accurate than random guessing.

*4. Summarize at least **two** assumptions of logistic regression.*

One assumption of logistic regression is that there is no multicollinearity between the independent variables, meaning the variables should not be highly correlated with one another.

Another assumption of logistic regression is a linear relationship between each independent variable and the log-odds of the outcome.

5. Provide evidence that the assumptions from part D4 were verified by providing either a code snippet or a screenshot.

This section of code finds the Variance Inflation Factor to determine multicollinearity between variables

```
#Ensure there is no multicollinearity in the data
from statsmodels.stats.outliers_influence import variance_inflation_factor

x_check = x_train[['SquareFootage', 'NumBathrooms', 'NumBedrooms', 'RenovationQualityRate']]
vif_df = pd.DataFrame({
    'Feature': x_check.columns,
    'VIF': [variance_inflation_factor(x_check.values, i)
           for i in range(x_check.shape[1])]
})
print(vif_df)
```

This section of code uses the Box-Tidwell test to check whether the relationship between the predictor and its log-odds are linear.

```
#Ensure linearity between the independent variables and the log-odds of the outcomes via Box-Tidwell test
x_check = x_train[['SquareFootage', 'NumBathrooms', 'NumBedrooms', 'RenovationQualityRate']].copy()

#Create interaction terms of each variable with its log
for col in x_check.columns:
    x_check[f'{col}_log'] = x_check[col] * np.log(x_check[col] + 1)

x_bt = sm.add_constant(x_check)
bt_model = sm.Logit(y_train, x_bt).fit(max_iter=1000)
print(bt_model.summary())
```

6. Discuss a limitation of your analysis.

A limitation of the analysis that is very notable is the issue of linearity seen in the results of the Box-Tidwell Test. Below is a screenshot of the output from the Box-Tidwell test.

Logit Regression Results						
=====						
Dep. Variable:	IsLuxury	No. Observations:	5451			
Model:	Logit	Df Residuals:	5442			
Method:	MLE	Df Model:	8			
Date:	Mon, 11 May 2026	Pseudo R-squ.:	0.2267			
Time:	09:34:39	Log-Likelihood:	-2921.6			
converged:	True	LL-Null:	-3778.2			
Covariance Type:	nonrobust	LLR p-value:	0.000			
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-6.8094	0.634	-10.732	0.000	-8.053	-5.566
SquareFootage	0.0132	0.003	3.852	0.000	0.006	0.020
NumBathrooms	1.3366	0.294	4.545	0.000	0.760	1.913
NumBedrooms	0.3515	0.344	1.021	0.307	-0.324	1.027
RenovationQualityRate	-0.2889	0.177	-1.633	0.103	-0.636	0.058
SquareFootage_log	-0.0014	0.000	-3.323	0.001	-0.002	-0.001
NumBathrooms_log	-0.4277	0.156	-2.737	0.006	-0.734	-0.121
NumBedrooms_log	0.1609	0.164	0.983	0.326	-0.160	0.482
RenovationQualityRate_log	0.1590	0.069	2.316	0.021	0.024	0.294
=====						

The interaction p-values of the SquareFootage_log, NumBathrooms_log, and RenovationQualityRate_log are all below 0.05, which implies that they are non-linear. Now, this does make some sense when you consider the real world meaning of each variable. For example, going from 1,000 square feet to 2,000 square feet probably has a bigger effect on determining luxury status than going from 5,000 to 6,000, so seeing a curve in that data is not surprising. The same logic can be applied when considering the number of bathrooms. Going from 1 bathroom to 2 bathrooms will have a bigger effect than going from 5 bathrooms to 6 bathrooms. This is a limitation to the analysis, but I do not consider the model itself to be flawed. Despite 3 of my 4 variables violating the linearity assumption, the model still achieved 74% accuracy on test data, telling me that the model can still be useful in predicting the luxury status of properties.

7. Recommend a course of action for the real-world organizational situation from part A1 based on your results.

Based on the results of the logistic regression, I would recommend the real estate professionals prioritize square footage, number of bedrooms and bathrooms, and renovation quality when attempting to assess, price, or market luxury properties. These were the four statistically significant variables when determining luxury status and these qualities can be highlighted in marketing efforts.

F. Acknowledge reference sources used to support the Python or R code application. All references listed should also include an in-text citation in the code annotation. Be sure the sources are reliable. If no sources were used for coding, state, "No sources used."

No Sources Used

G. Acknowledge sources, using in-text citations and references, for content that is quoted, paraphrased, or summarized.

No Sources Used